

INSTITUTO TECNOLÓGICO NACIONAL DE MÉXICO

CAMPUS VERACRUZ

Título del Trabajo

Documentación de “Bazar Stereo”

Materia

Bases de Datos NoSQL

Docente

Jose Nahum Prieto

Integrantes

Martínez Carreola Daniel Alejandro

Ruíz de la Cruz César

Tabla de Contenido

Introducción **3**
Diagrama Entidad - Relación **5**
Diagrama Relacional **5**
Diagrama de Documentos **6**
Consultas CRUD 7
Pantallas de la Página Web **9**
Página de Inicio 9
Inicio de Sesión 10
Registro como nuevo usuario 11
Discos a la venta 11
Perfil de Usuario 12
Editar datos del perfil 13
Panel de administrador 14
Gestión de usuarios 14
Gestión de discos 15
Estadísticas de venta 15
Panel de vendedor 16
Menú de subastas 16
Detalle de una subasta 17
Detalle de un disco 17
Panel para ofertar en subasta 18
Subasta no exitosa 18
Correo de confirmación de compra/ subasta 19
Conclusión **20**

Introducción

Bazar-Stereo es un ecommerce tipo marketplace orientado a la compraventa de discos de música en formatos físicos, principalmente CD y vinilo. La plataforma surge originalmente en la materia de Programación Web con una base de datos relacional en PostgreSQL; posteriormente fue exportada a Supabase y, en la versión actual del proyecto, se migra a MongoDB Atlas.

A diferencia de un ecommerce tradicional, Bazar-Stereo permite que los propios usuarios abastezcan el catálogo y actúen tanto como compradores como vendedores. Además de la venta convencional, la plataforma incorpora subastas para artículos considerados de edición limitada o coleccionables.

Alcance del proyecto:

Discos : El producto central de la plataforma serán los discos de música en formatos físicos (cd, vinilo), los cuales serán registrados de forma única por cada usuario que los vende, y tendrá una gran flexibilidad en el detalle que se le puede dar a un artículo publicado.

Catálogo de Discos : El catálogo de productos estará dividido en dos secciones principales, en las cuales se mostrarán : Discos que se encuentren a la venta de forma convencional, y discos que se encuentren en subastas abiertas.

Filtrado de catálogo : Las secciones del catálogo podrán ser filtradas por : género musical al que pertenece el disco y formato (cd o vinilo).

Ventas : La compra y venta de productos dentro de la plataforma estará limitada únicamente a usuarios con perfiles registrados.

Subastas : Alterno a la venta de un disco, determinados artículos considerados en el mercado como de “edición limitada” o “coleccionable” podrán ser subastados a partir de un precio base, con una duración determinada.

Usuarios : La plataforma no hará distinción entre vendedores (realizan ventas y subastas) y compradores (sólo realizan compras), permitiendo a un usuario participar en ambas actividades libremente.

Perfil de Usuario : La plataforma permitirá al usuario registrar datos personales y de envío mediante la gestión de su perfil, además, podrá visualizar su historial de compras hasta los últimos 30 días.

Administradores: El rol de los administradores de la plataforma será mantener la integridad del catálogo, siendo capaces de consultar y modificar los detalles de un artículo antes y después de su publicación.

Reglas del negocio

Para las **ventas**:

- Por cada venta realizada, la plataforma toma una comisión del 6% del precio del artículo.

Para las **subastas**:

- El vendedor deberá determinar un precio base para su artículo, y las pujas se realizarán en porcentajes de este mismo.

- Por cada subasta finalizada exitosamente se cobrará el 11% del precio final del artículo si este no supera los $20,000.

- Si el precio final del artículo subastado supera los $20,000, la plataforma toma una comisión del 16%.

- Al pujar por un artículo, solamente se puede hacer en porcentajes preestablecidos por la plataforma, y el monto será calculado al momento.

- El vendedor puede determinar un precio mínimo oculto que deberá ser alcanzado por las pujas, o de lo contrario, el producto no se venderá y no se cobrará comisión.

- Las nuevas pujas deberán superar en al menos 5% a la puja anterior.

- Las pujas pueden ser canceladas hasta máximo una hora antes de que se cierre la subasta.

Diagrama Entidad - Relación

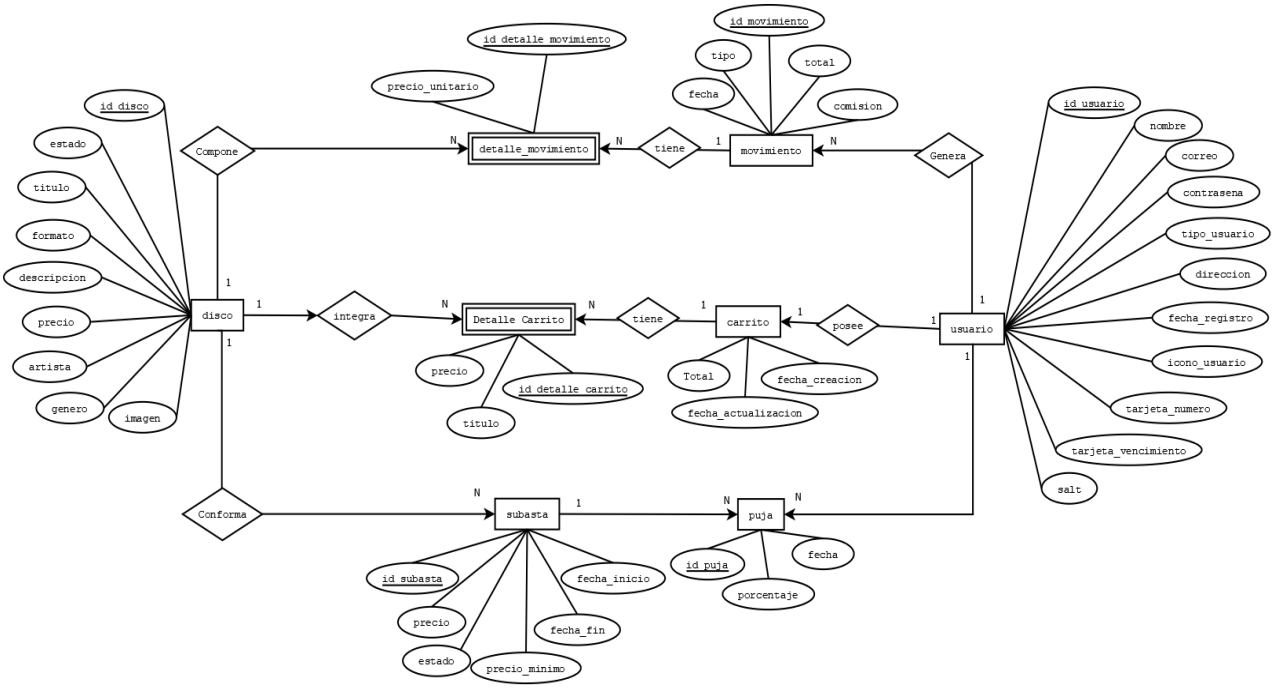


Diagrama Relacional

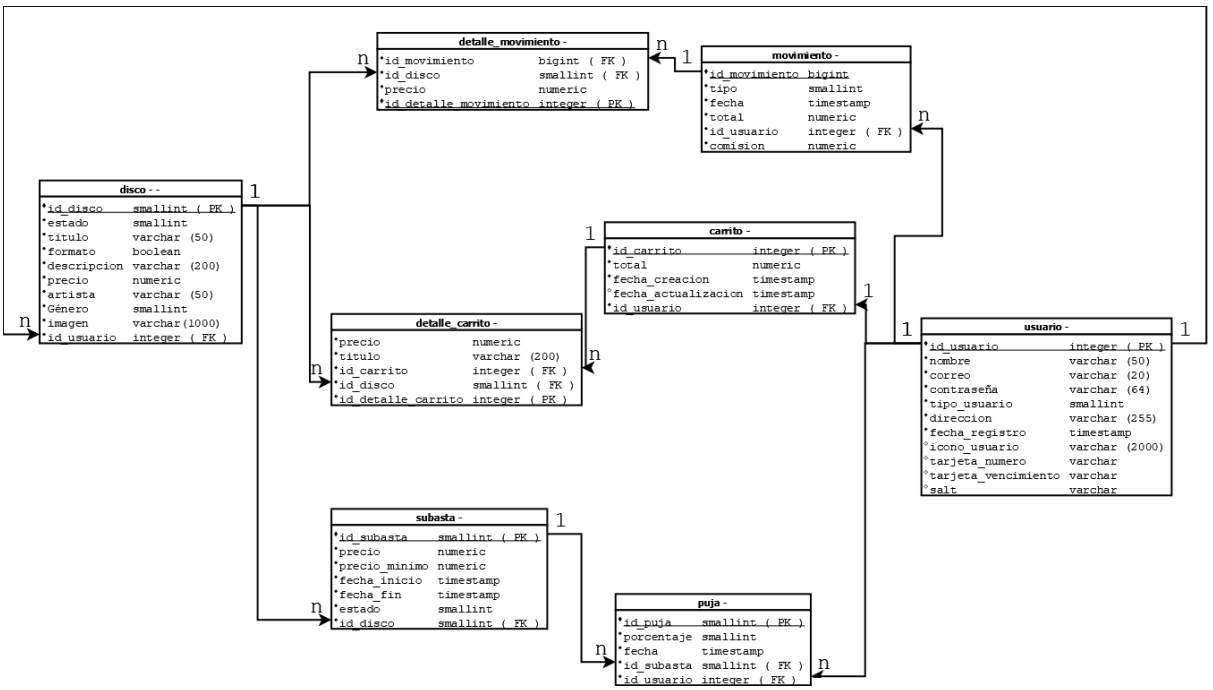
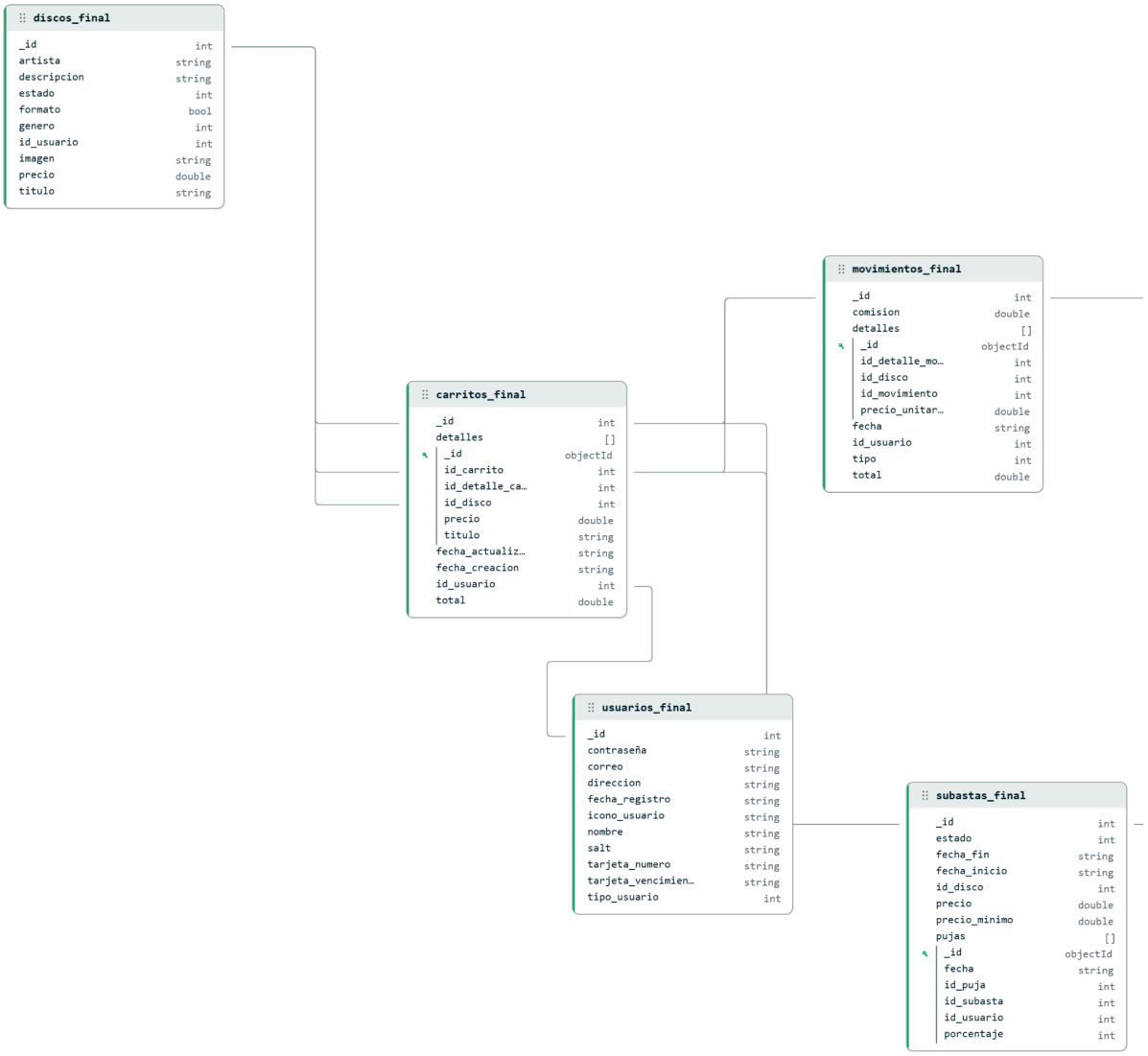


Diagrama de Documentos



Consultas CRUD

Autenticación	
Consulta	Propósito
Usuario.objects(correo__iexact=correo).first()	Buscar un usuario por correo electrónico sin distinguir mayúsculas y minúsculas.
Usuario.objects(correo=correo).first()	Verificar si un correo ya está registrado.
Usuario.objects.order_by("-id_usuario").first()	Obtener el usuario con el ID más alto para calcular el siguiente identificador.
nuevo.save()	Guardar un nuevo usuario en la base de datos.
usuario.save()	Guardar los cambios en los datos del usuario, incluyendo la actualización del hash y salt de una contraseña antigua.

Admin	
Consulta	Propósito
Usuario.objects.count()	Contar los usuarios registrados.
Movimiento.objects.count()	Contar los movimientos registrados.
Movimiento.objects.only("fecha", "total", "comision")	Consultar únicamente los campos necesarios para calcular estadísticas de ventas e ingresos.
Usuario.objects.order_by("-fecha_registro")	Obtener los usuarios ordenados desde el registro más reciente.
Usuario.objects(id_usuario=id).first()	Buscar un usuario específico por su identificador.
usuario.delete()	Eliminar el documento del usuario encontrado.
Movimiento.objects.order_by("-fecha").limit(100)	Consultar los 100 movimientos más recientes.
Disco.objects(id_disco__in=list(ids_discos))	Buscar los discos cuyos identificadores aparecen en los movimientos consultados.
Usuario.objects(id_usuario__in=list(vendedores_agg.keys()))	Consultar los usuarios que corresponden a los vendedores identificados en las estadísticas.

Disco.objects.order_by("-id_disco")	Consultar los discos ordenados por identificador descendente.
Disco.objects(id_disco=id).first()	Buscar un disco por su identificador antes de eliminarlo.
disco.delete()	Eliminar el documento del disco seleccionado.

discos	
Consulta	Propósito
Disco.objects(estado=0)	Consultar los discos disponibles para la venta.
Disco.objects(estado=0, id_usuario__ne=current_user.id_usuario)	Consultar discos disponibles excluyendo los publicados por el usuario actual.
Disco.objects(formato__in=formatos)	Filtrar discos según los formatos seleccionados.
Disco.objects(genero__in=generos)	Filtrar discos según los géneros seleccionados.
query.count()	Contar los documentos que coinciden con los filtros aplicados.
query.skip(offset).limit(por_pagina)	Obtener un segmento de resultados para implementar la paginación.

carrito	
Consulta	Propósito
Carrito.objects(id_usuario=current_user.id_usuario).order_by("-fecha_creacion").first()	Obtener el carrito más reciente del usuario autenticado.
carrito.save()	Crear el carrito si es nuevo o guardar sus cambios.
Disco.objects(id_disco=id_disco, estado=0).first()	Buscar un disco específico que esté disponible para la venta.
Disco.objects(id_disco__in=disco_ids)	Consultar los discos que corresponden a los identificadores guardados en el carrito.
carrito.detalles.append(DetalleCarrito(id_detalle_carrito=nuevo_detalle_id, id_carrito=0, id_disco=disco.id_disco, precio=disco.precio, titulo=disco.titulo))	Agregar un disco a la lista de detalles del carrito.
Movimiento.objects.order_by("-id").first()	Obtener el movimiento con el ID más alto para calcular el siguiente identificador.
Movimiento(id=nuevo_id_mov, tipo=1, fecha=datetime.now(), total=total, id_usuario=current_user.id_usuario, comision=comision, detalles=nuevos_detalles).save()	Crear y guardar un movimiento correspondiente a una compra completa.
DetalleMovimiento(id_movimiento=nuevo_id_mov, id_disco=disco.id_disco, precio_unitario=det.precio)	Construir un detalle embebido que relaciona el movimiento con un disco y su precio.
disco.save()	Guardar el cambio de estado del disco cuando se vende.
carrito.detalles = [d for d in carrito.detalles if d.id_detalle_carrito != id_detalle]	Retirar del carrito el detalle cuyo identificador coincide con el indicado.
carrito.detalles = []; carrito.total = 0.0; carrito.save()	Vaciar el carrito y restablecer su total después de la compra.

detalle disco	
Consulta	Propósito
Disco.objects(id_disco=id).first()	Buscar el disco que se desea mostrar.
Usuario.objects(id_usuario=disco.id_usuario).first()	Obtener la información del vendedor del disco.
Disco.objects(id_usuario=disco.id_usuario).only("id_disco")	Obtener los identificadores de los discos publicados por ese vendedor.
Movimiento.objects(detalles__id_disco__in=discos_ids).distinct("id_movimiento")	Consultar los identificadores distintos de movimientos que contienen discos del vendedor.

detalle subasta	
Consulta	Propósito
Subastas.objects(id=id_subasta).first()	Buscar la subasta por su identificador.
Disco.objects(id_disco=subasta.id_disco).first()	Obtener el disco asociado con la subasta.
Usuario.objects(id_usuario=disco.id_usuario).first()	Buscar al vendedor propietario del disco.
Disco.objects(id_usuario=disco.id_usuario).only("id_disco")	Obtener los identificadores de los discos del vendedor.
Movimiento.objects(detalles__id_disco__in=discos_ids).distinct("id")	Consultar los movimientos distintos relacionados con esos discos.
Usuario.objects(id_usuario=puja_maxima.id_usuario).first()	Obtener al usuario que realizó la puja máxima, cuando existe una puja máxima.
Usuario.objects(id_usuario__in=usuarios_ids)	Consultar los usuarios cuyos identificadores están en la lista proporcionada.
Movimiento.objects.order_by("-id").first()	Obtener el movimiento con el ID más alto para generar el siguiente.
subasta.pujas.append(Puja(...))	Guardar el movimiento de compra generado al finalizar una subasta.
movimiento.save()	Agregar una puja a la lista de pujas embebidas de la subasta.
subasta.save()	Guardar las pujas o los cambios en el estado de la subasta.

disco.save()	Guardar el cambio de estado del disco asociado a la subasta.

mi cuenta	
Consulta	Propósito
Movimiento.objects(...).count()	Contar los movimientos asociados al usuario.
Subastas.objects(estado__ne=1).all()	Consultar las subastas cuyo estado no es 1 (no están activas).
Movimiento.objects(...).order_by("-id").all()	Consultar el historial de movimientos ordenado desde el más reciente.
Disco.objects(...).only(...)	Consultar determinados campos de los discos, evitando recuperar todos sus atributos.
Usuario.objects(id_usuario=...).first()	Buscar el registro de un usuario por su identificador.
usuario.save()	Guardar los cambios realizados en el perfil, contraseña, datos de tarjeta o imagen del usuario.

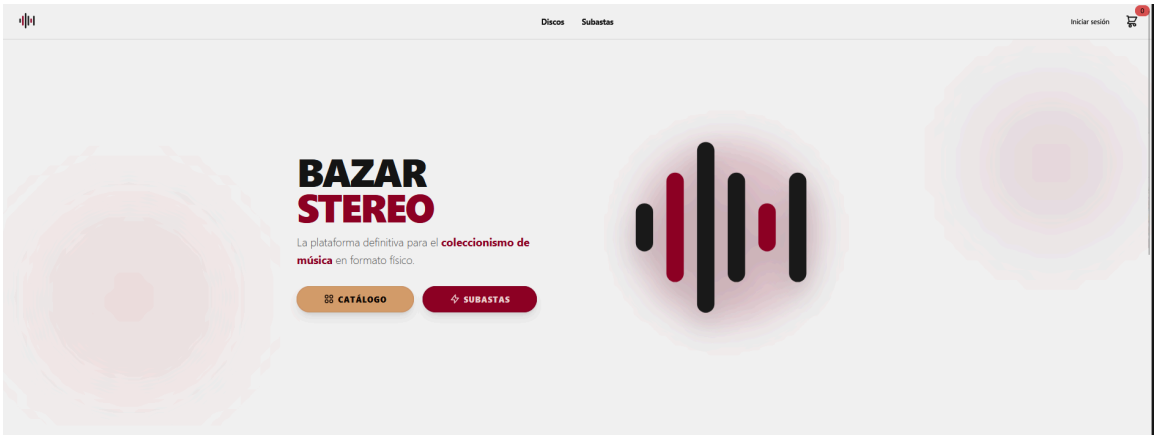
subastas	
Consulta	Propósito
Subastas.objects(estado=1)	Consultar las subastas activas.
Subastas.objects(estado=1).only("id_disco")	Obtener únicamente los identificadores de los discos asociados a subastas activas.
Disco.objects(id_disco__in=discos_ids)	Consultar los discos asociados con los identificadores obtenidos.
Disco.objects(id_usuario__ne=current_user.id_usuario)	Excluir los discos publicados por el usuario actual.
Disco.objects(formato__in=formatos)	Filtrar los discos por formato.
Disco.objects(genero__in=generos)	Filtrar los discos por género musical.
query.count()	Contar los resultados de la consulta.
query.skip(offset).limit(por_pagina)	Recuperar únicamente los resultados

	correspondientes a una página del catálogo.

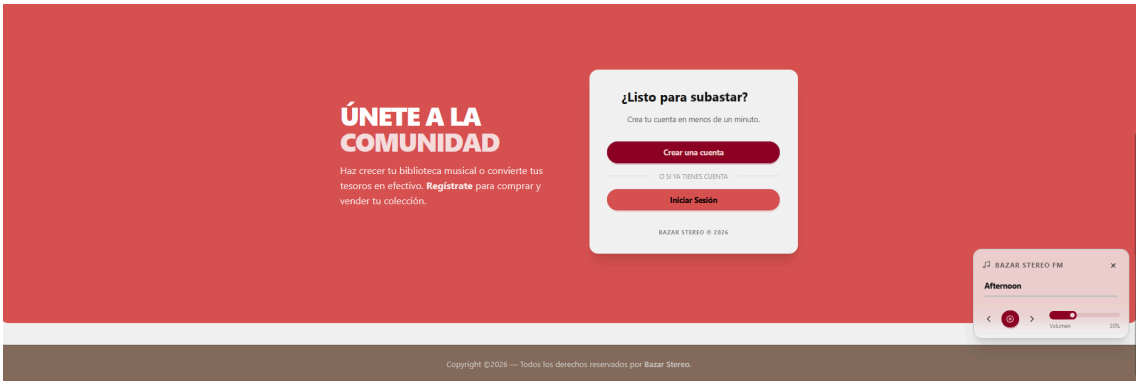
vendedor	
Consulta	Propósito
Disco.objects(id_usuario=...).all()	Consultar los discos publicados por un vendedor.
Subastas.objects(...)	Consultar las subastas correspondientes a los criterios establecidos por el controlador.
Movimiento.objects(...)	Consultar los movimientos relacionados con las ventas del vendedor.
Usuario.objects(...)	Buscar los usuarios relacionados con los movimientos consultados.
query.count()	Contar los registros que coinciden con una consulta.
query.order_by(...).first()	Obtener el primer registro después de aplicar el orden especificado.
Disco(...).save()	Crear y guardar un disco publicado por el vendedor.
Subastas(...).save()	Crear y guardar una subasta.
disco.save()	Guardar los cambios de un disco.
subasta.save()	Guardar los cambios de una subasta.
disco.delete()	Eliminar un disco.

Pantallas de la Página Web

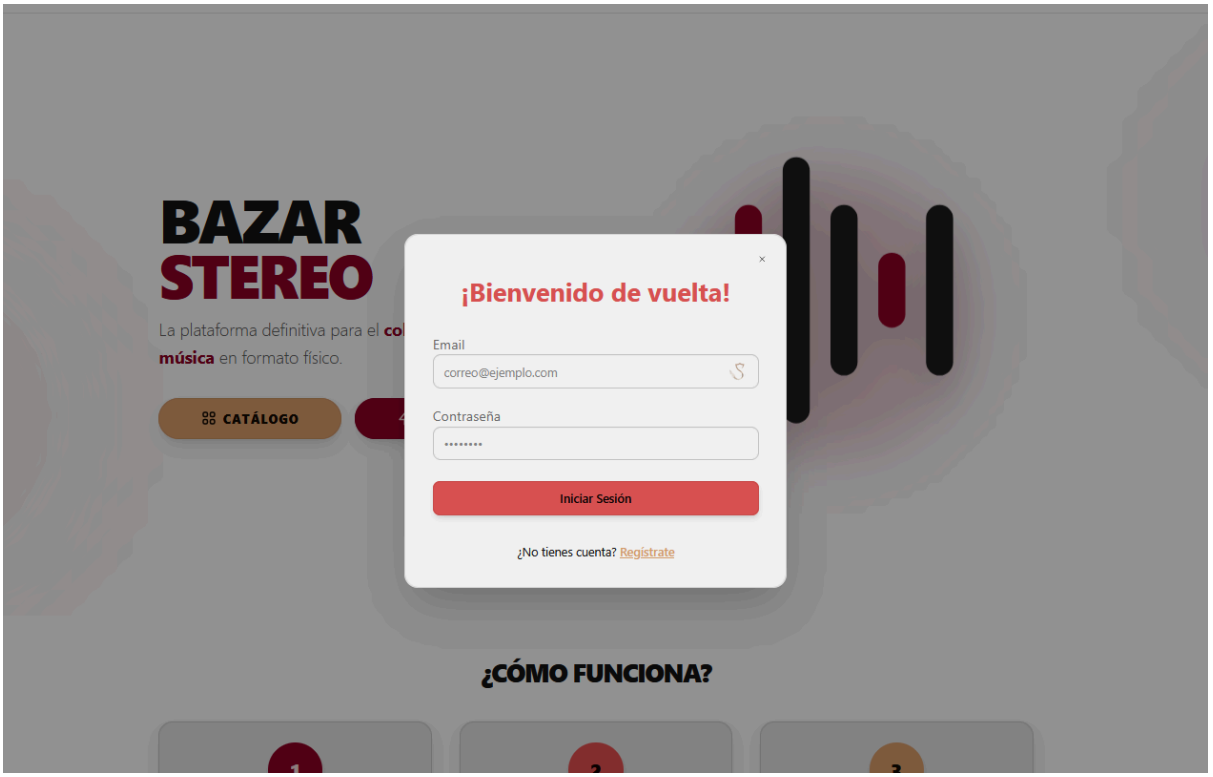
Página de Inicio



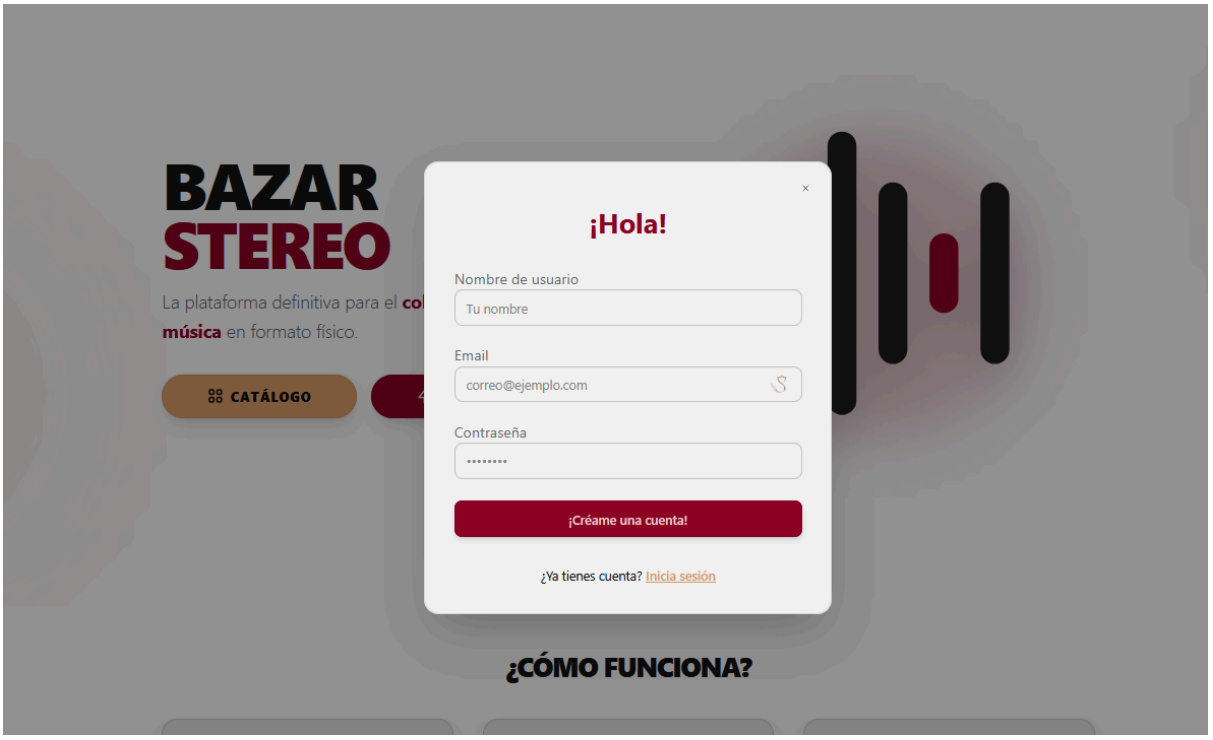




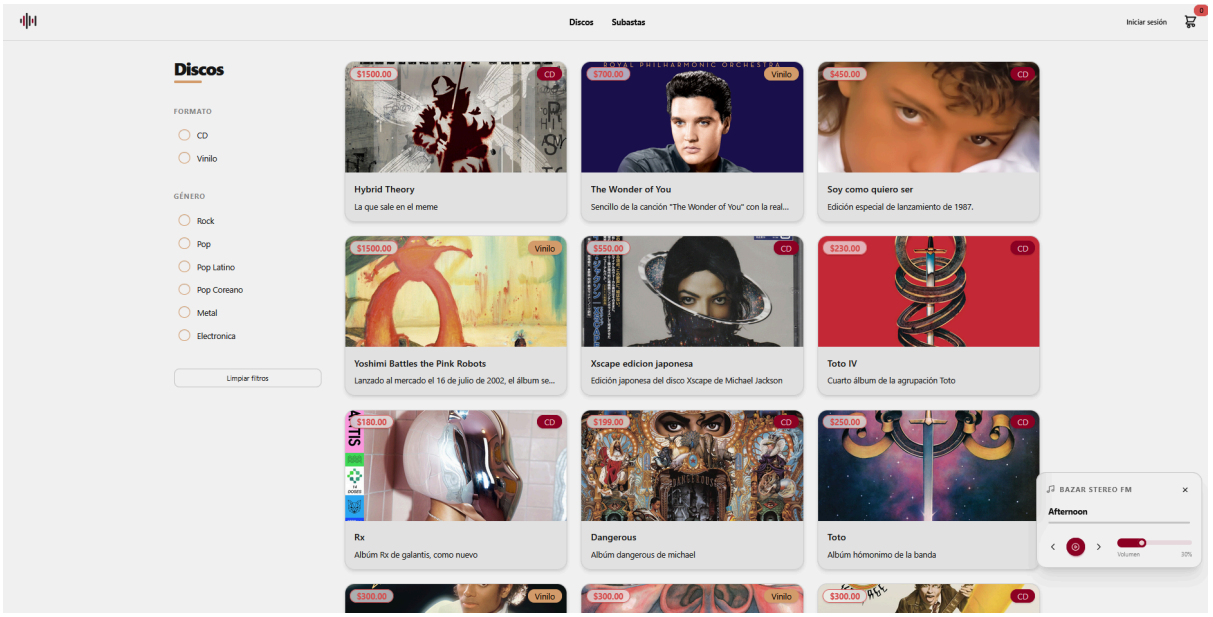
Inicio de Sesión



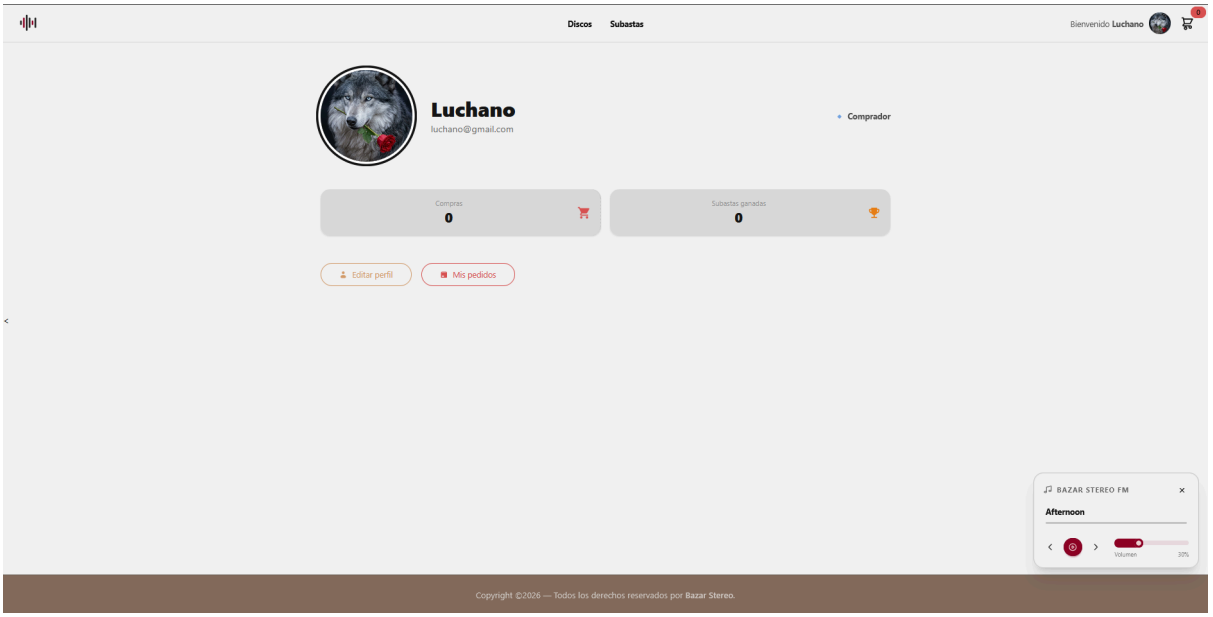
Registro como nuevo usuario



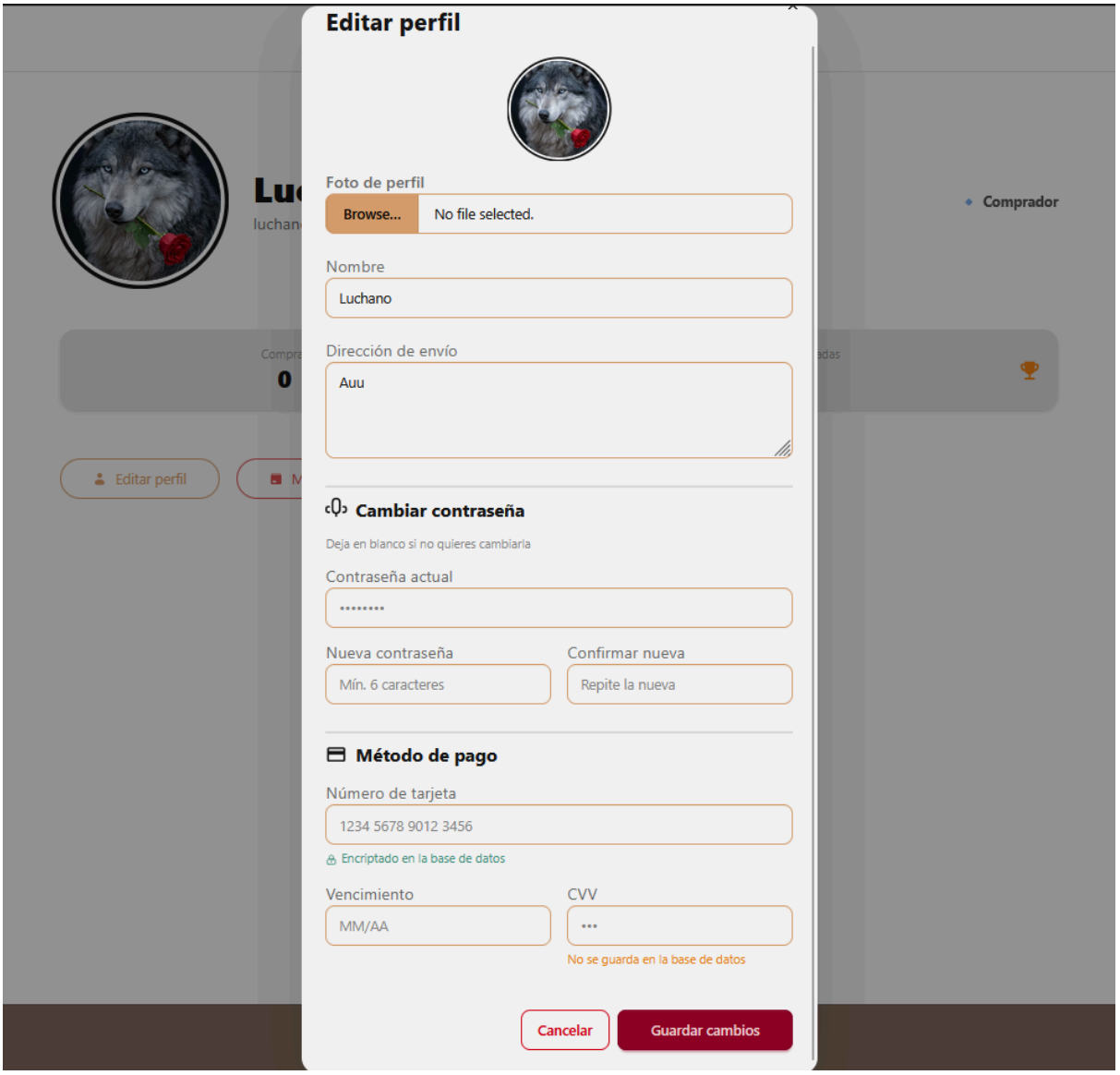
Discos a la venta



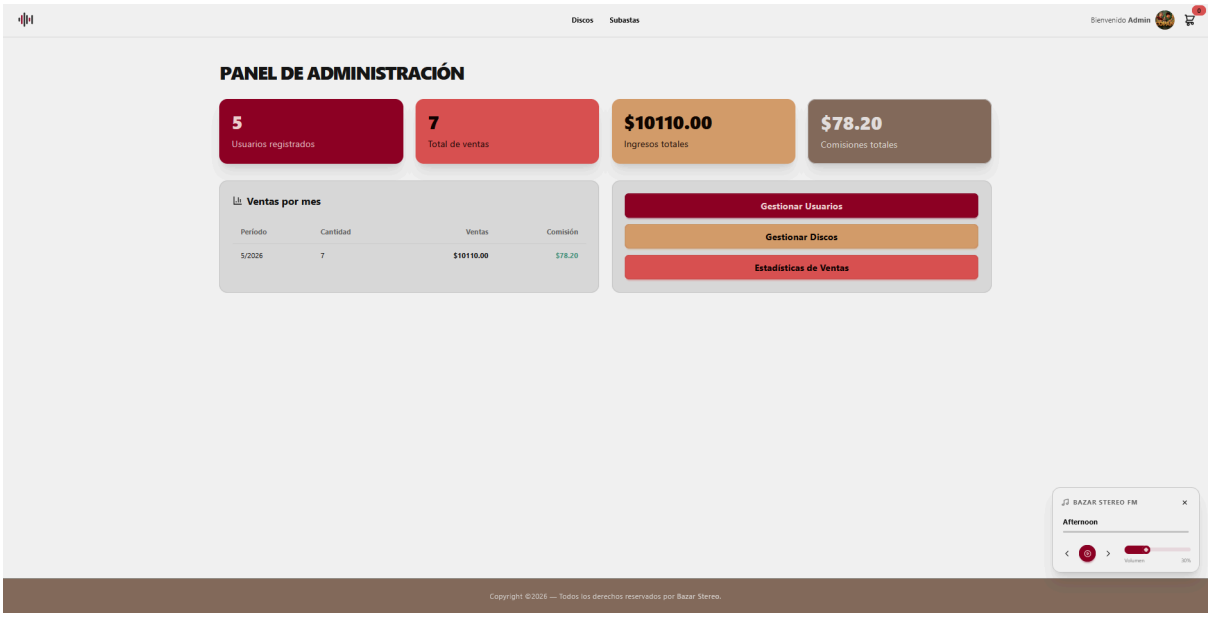
Perfil de Usuario



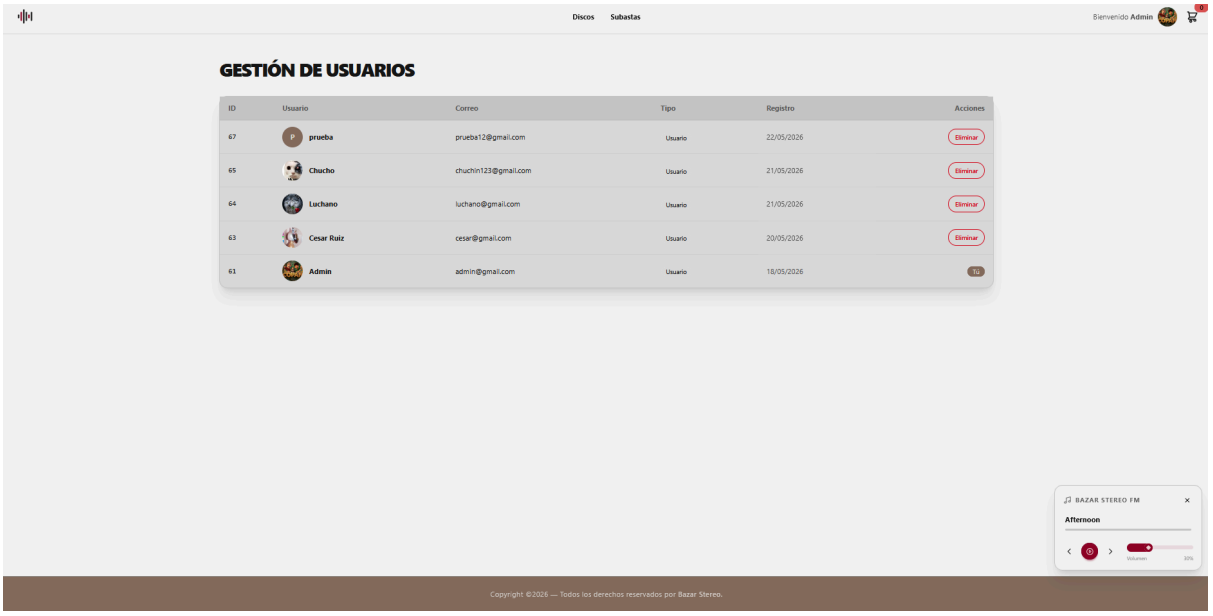
Editar datos del perfil



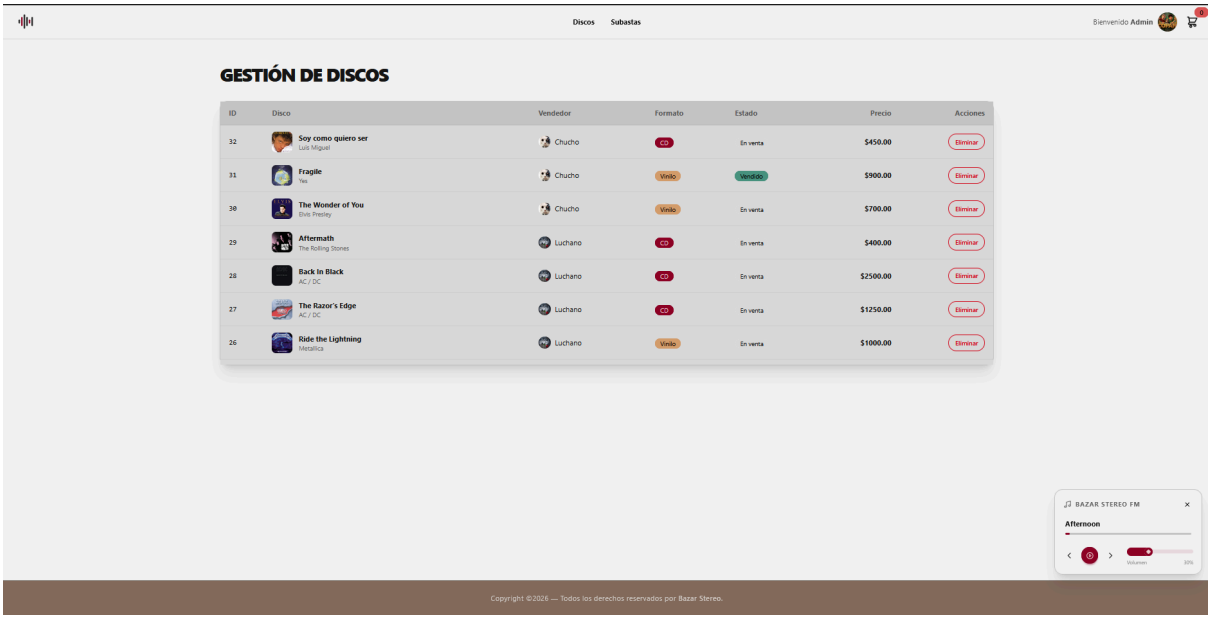
Panel de administrador



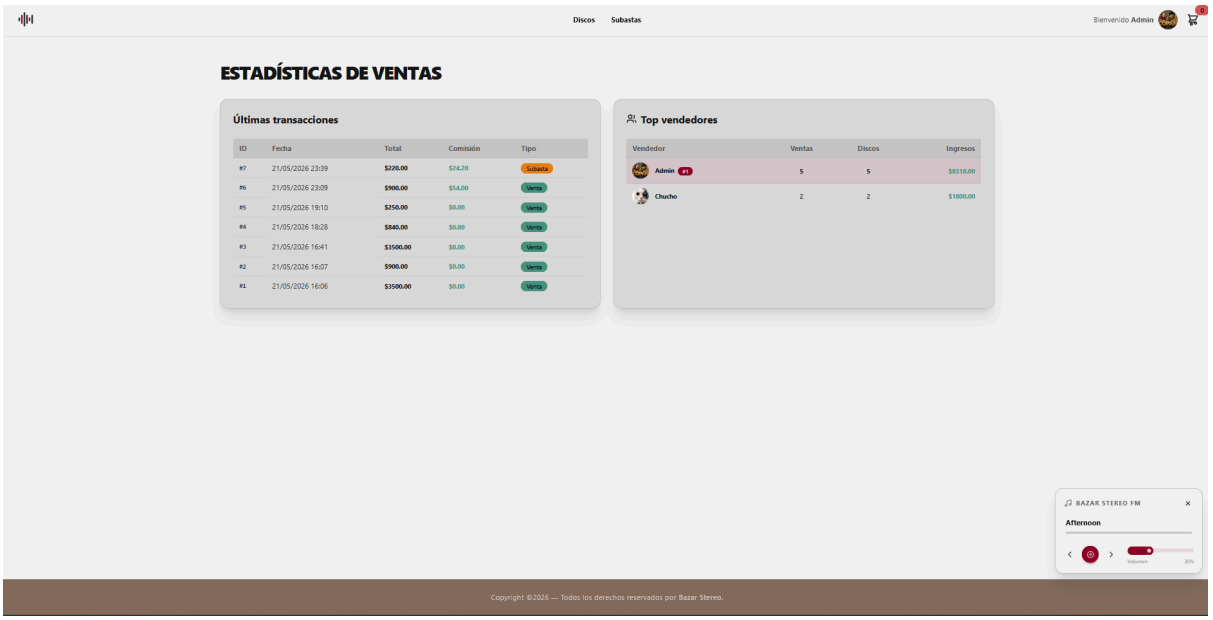
Gestión de usuarios



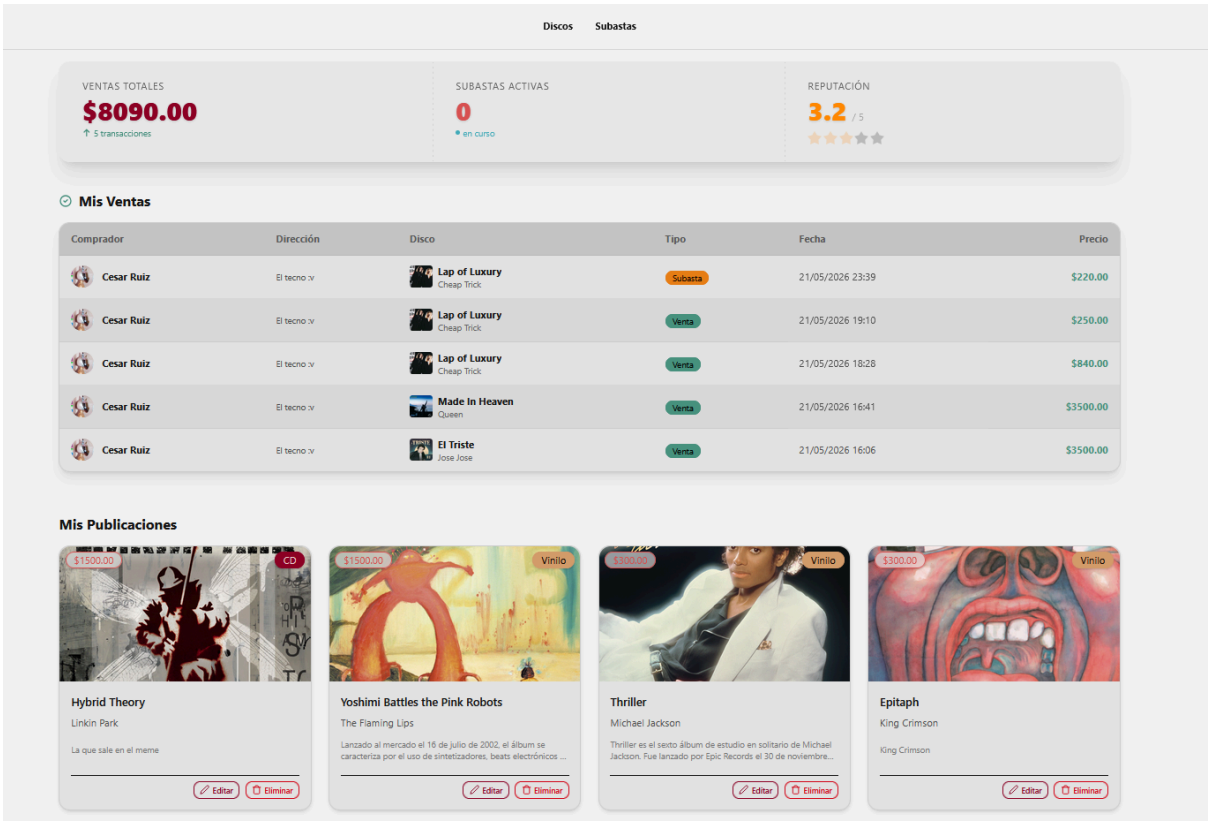
Gestión de discos



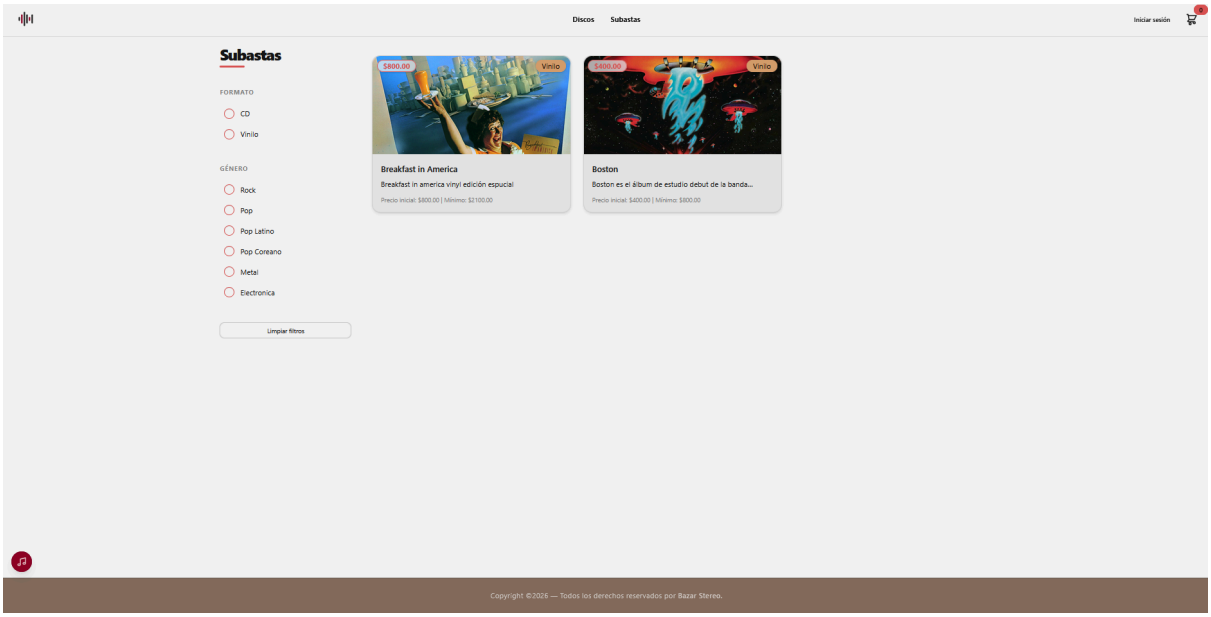
Estadísticas de venta



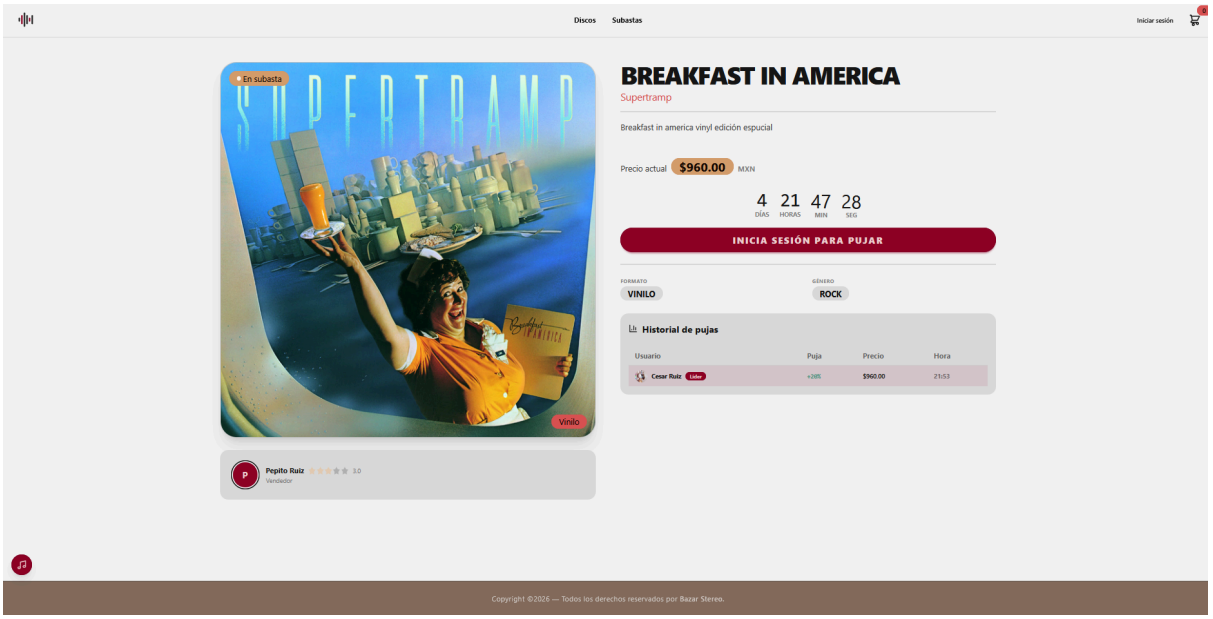
Panel de vendedor



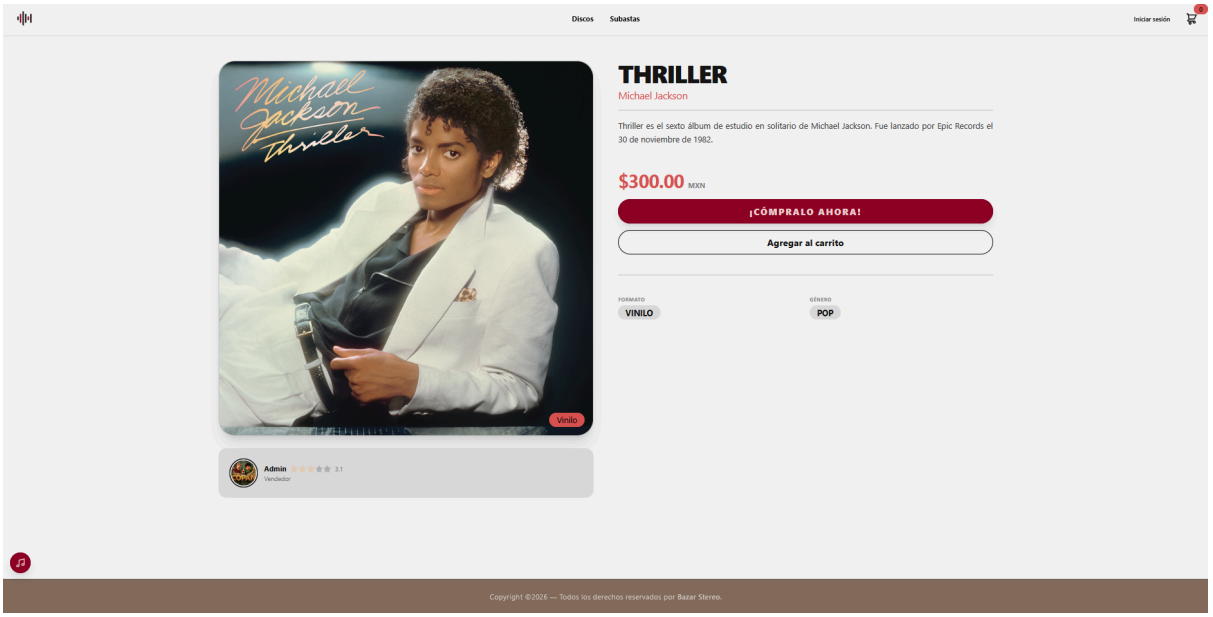
Menú de subastas



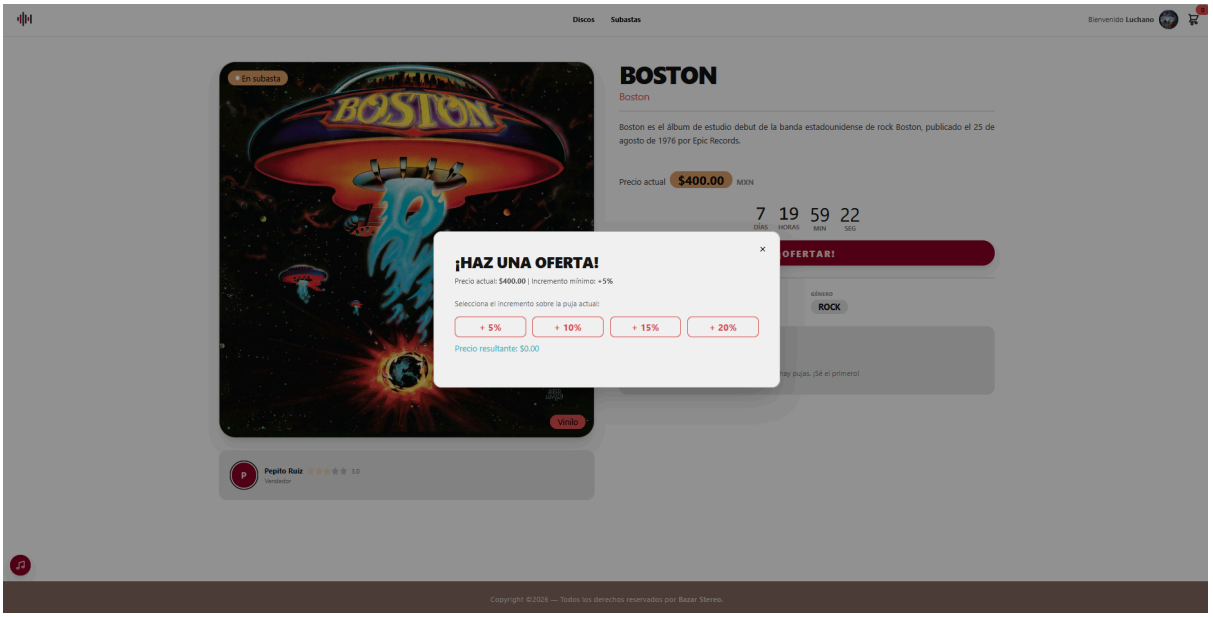
Detalle de una subasta



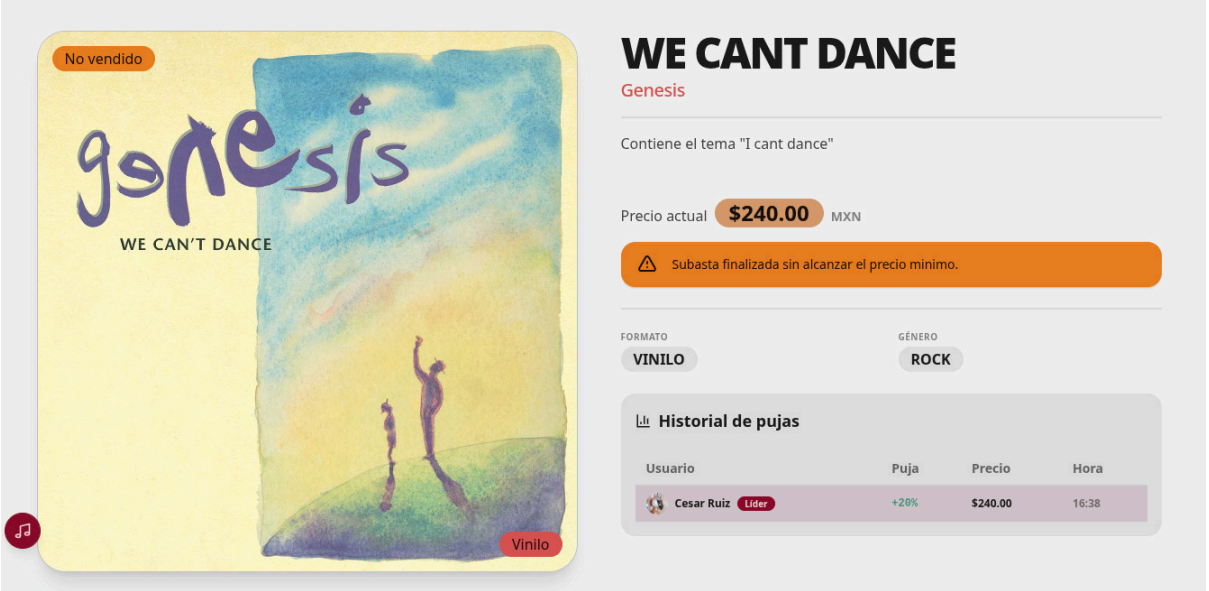
Detalle de un disco



Panel para ofertar en subasta

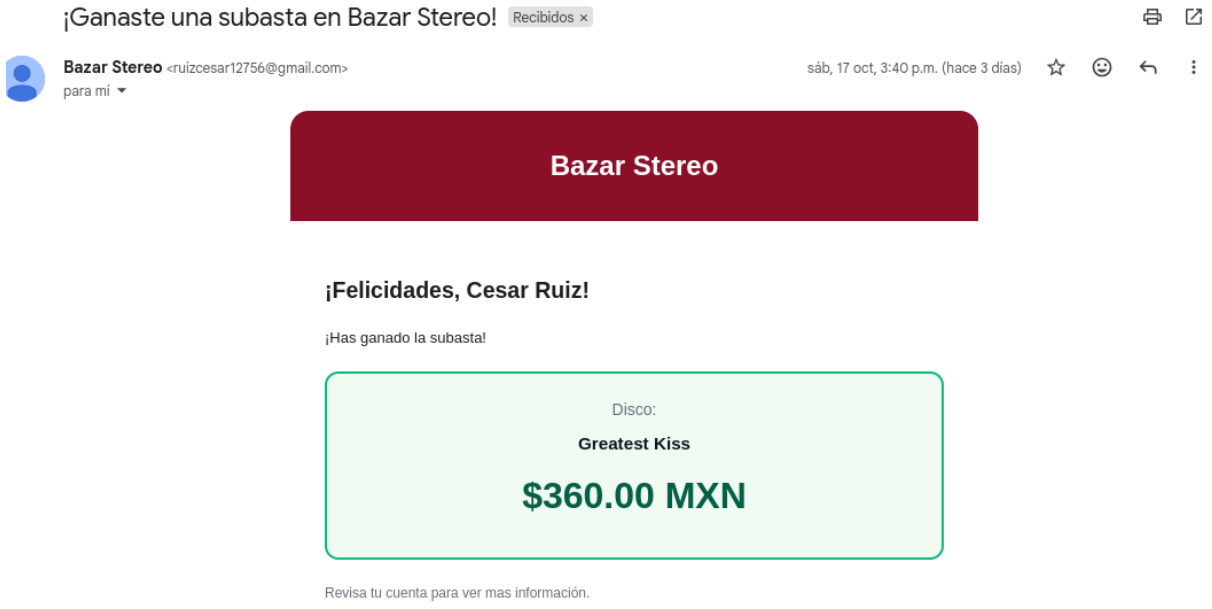


Subasta no exitosa



Correo de confirmación de compra/ subasta





Conclusión

Tras la realización de este proyecto, pudimos adaptar la base de datos relacional de nuestro proyecto a una base de datos no relacional en mongoDB, de lo que no solamente aprendimos la diferencia entre un modelo y otro, sino que también fue necesario adaptar las consultas para funcionar con la nueva base de datos.

Aparte de aprender a usar una base de datos no relacional en un proyecto, también aprendimos algunas herramientas que nos permitieron llevar nuestro proyecto un poco más lejos, desplegando nuestra aplicación en un servicio como Heroku y también obteniendo un dominio para acceder más fácilmente mediante name.com , todo esto aprovechando los apoyos que github le ofrece a los estudiantes.